

# LUT-DLA: Lookup Table as Efficient Extreme Low-Bit Deep Learning Accelerator

Guoyu Li<sup>††#\*</sup>, Shengyu Ye<sup>†#\*</sup>, Chunyun Chen<sup>§\*</sup>, Yang Wang<sup>†§</sup>, Fan Yang<sup>†</sup>, Ting Cao<sup>†</sup>,

Cheng Liu<sup>†</sup>, Mohamed M. Sabry Aly<sup>§</sup> and Mao Yang<sup>†</sup>

Microsoft Research<sup>†</sup>, University of Chinese Academy of Sciences<sup>†</sup>, NTU Singapore<sup>§</sup>

liguoyu21@mails.uca.ac.cn, {v-shengyue, yang.wang92, fanyang, ting.cao, maoyang}@microsoft.com,

chunyun001@e.ntu.edu.sg, liucheng@ict.ac.cn, msabry@ntu.edu.sg

**Abstract**—The emergence of neural network capabilities invariably leads to a significant surge in computational demands due to expanding model sizes and increased computational complexity. To reduce model size and lower inference costs, recent research has focused on simplifying models and designing hardware accelerators using low-bit quantization. However, due to numerical representation limits, scalar quantization cannot reduce bit width lower than 1-bit, diminishing its benefits. To break through these limitations, we introduce LUT-DLA, a Look-Up Table (LUT) Deep Learning Accelerator Framework that utilizes vector quantization to convert neural network models into LUTs, achieving extreme low-bit quantization. The LUT-DLA framework facilitates efficient and cost-effective hardware accelerator designs and supports the LUTBoost algorithm, which helps to transform various DNN models into LUT-based models via multistage training, drastically cutting both computational and hardware overhead. Additionally, through co-design space exploration, LUT-DLA assesses the impact of various model and hardware parameters to fine-tune hardware configurations for different application scenarios, optimizing performance and efficiency. Our comprehensive experiments show that LUT-DLA achieves improvements in power efficiency and area efficiency with gains of 1.4~7.0 $\times$  and 1.5~146.1 $\times$ , respectively, while maintaining only a modest accuracy drop. For CNNs, accuracy decreases by 0.1%~3.1% using the  $L_2$  distance similarity, 0.1%~3.4% with the  $L_1$  distance similarity, and 0.1%~3.8% when employing the Chebyshev distance similarity. For transformer-based models, the accuracy drop ranges from 1.4% to 3.0%.

## I. INTRODUCTION

The growth of neural network models demonstrates extraordinary abilities, and *Neural Network Scaling Law* [24], [30] unveils a fundamental relation between the number of model parameters, computational costs, and model capabilities. Therefore, the efficient inference of larger-scale models has become a crucial research topic. An essential and typical optimization approach is to employ lower-precision quantization for efficient model inference and representation.

Recent studies have quantized Large Language Models (LLMs) to FP8 [39] [40], FP4 [36] [18], INT2 [7], [57] and even to 1.58 bits [37] and 1 bits [59], substantially shrinking the model scale and memory footprint. Additionally, hardware accelerators have made progress in supporting low-bit quantization computation in ALUs. For instance, NVIDIA's

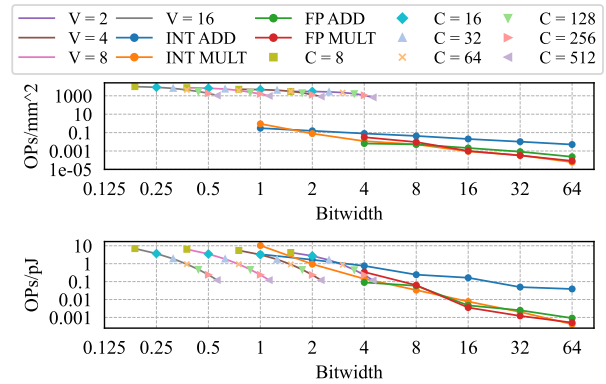


Fig. 1: Comparison of Area and Power Efficiency: LUT-Based Approximate Computing vs. ALU (**higher is better**, 28 nm FD-SOI@300 Mhz,  $1k \times 1k \times 1k$  matrix multiplication,  $V$ =vector length,  $C$ =number of centroids, equivalent bitwidth= $V/\log_2 C$ )

Blackwell [46], Hopper [8] and Turing Tensor Cores [5] now enable operations in FP8, FP4, and 1-bit, respectively. Fig. 1 collects area efficiency (Ops/ $\mu m^2$ ) and power efficiency (Ops/pJ) of floating point and integer operations with various bitwidths. It illustrates how scaling lower-bit computations can significantly enhance processing capabilities without increasing area cost or energy consumption.

**However, due to the inherent limitations of numerical representation, the benefits of scaling down bit width in hardware no longer exist**, and thus scalar quantization methods cannot compress data representation to less than 1-bit. Fig. 1 indicates that the current quantization approach has hit its limit, resulting in a 1-bit numerical representation. At reduced bitwidths, area and power efficiency for different operators converge. Ultimately, the current quantization approach reaches the limit of 1-bit precision, restricting the corresponding computational resources to bit operations made up of a few logic gates, which complicates further simplification. It suggests that existing quantization methods and hardware architectures have reached their computing and power efficiency limits [12] [31] [23].

The recent **Lookup Table (LUT)-based model architecture** [1], [4], [32], [50], [55] introduces a novel computing

\* These authors contributed equally to this work

# Contribution during internship at Microsoft Research

§ Corresponding author

paradigm for neural networks. It employs **Vector Quantization** to leverage the semantic similarity of feature maps (activations). LUT-based model method uses a single index to represent a vector, which simplifies the representation of input features as limited feature vectors (centroids). During inference, the interaction between feature vectors and specific weights is static when performing matrix multiplication or convolution operators. Therefore, LUT-based models can precompute the computational results of features and weights and store them in LUTs.

LUT-based models effectively turn operations that require a lot of computing power into small, efficient LUTs, which greatly reduces the amount of extra computing and memory needed, addressing the limitations of conventional accelerators [12]. Fig. 1 illustrates that employing LUT-based models can enhance computational efficiency by 1~5 orders of magnitude and power efficiency by 1~2 orders of magnitude compared to traditional ALUs.

However, the current LUT-based model also needs to be revised for hardware architectures. **First**, LUT-based models operate within a Memory-Centric Computing framework [43], necessitating improved hardware for LUT access. While previous research has addressed architectural limits [1], [32], [55], none have addressed the difficulty of on-chip data reuse. **Second**, previous LUT-based model training require starting from scratch, which is very time-consuming and difficult to converge. Consequently, previous works [1], [50], [52] are often limited to small models. There is an urgent need for a lightweight model conversion algorithm to increase conversion efficiency and to explore simpler similarity algorithms to further reduce inference overhead. **Third**, LUT-based model accelerator design involves a sophisticated interplay of numerous hyperparameters that impact not only model accuracy but also hardware efficiency and overall system cost. An efficient and targeted design methodology is essential for co-optimizing software and hardware parameters.

In this paper, we propose LUT-based Extreme Low-Bit Deep Learning Accelerator (DLA) framework. Our contributions are organized as follows:

- We design a **flexible parameterized hardware accelerator generator LUT-DLA**. The architecture not only accelerates LUT-based model feature comparisons and table lookups but also facilitates an in-depth analysis of hardware costs and dataflow trade-offs, ensuring optimal configuration within LUT-DLA.
- We design a **lightweight multistage algorithm LUTBoost** to transform the model into LUT-based model, which enhances training convergence, accuracy, and stability. LUTBoost also advances the similarity computation in LUT-based model, introducing a novel design space dimension to LUT-DLA that finely balances computational accuracy with hardware cost.
- We conduct a **Co-Design Space Exploration Engine** to evaluate the interaction between hardware and software parameters and effects on both the cost-efficiency of hardware architecture and model accuracy. The engine optimizes

the exploration space, accelerating the identification and selection of suitable hardware and algorithm configurations for agile and efficient deployments.

- **We evaluate LUT-DLA end-to-end performance on CNNs and Transformer-based models.** The results indicate that LUT-DLA is 6.2~12.0 $\times$  faster than traditional DLAs under the same area with a modest compromise in accuracy. For CNNs, a reduction of 0.1% to 3.1% is observed with the  $L_2$  distance, 0.1% to 3.4% with the  $L_1$  distance, and 0.1% to 3.8% when utilizing the Chebyshev distance. Meanwhile, Transformer-based models experience a minor decrease in accuracy ranging from 1.4% to 3.0%.

## II. BACKGROUND AND MOTIVATION

### A. Approximate Computing in Neural Networks

Approximate computing [2], [21] is a promising approach that effectively simplifies computation, memory, and power consumption. Due to the abundance of similar features and redundant data in neural networks, approximate computation can significantly reduce the inference costs of neural networks. There are many works based on approximate computing, including *Precision Scaling/Quantization* [11], [17], [35], *Skipping* [22], [47], *Memorization* [33], [41], [51] and *Approximate Multipliers/Adders* [42], [56]. In contrast to algebraic operations, LUT provides a non-computational and flexible approach for function mapping, and is widely utilized in approximation computing. For example, to optimize quantized range mapping [60], reuse computational results [48], and even substitute GEMM computations by directly storing precomputed results [1], [4], [50], [55]. LUT can also substitute or simplify non-linear functions used in neural network activation operations, such as NN-LUT [61] based on polynomial approximation and TransPimLib [26], which combines CORDIC and LUT on PIM architectures. Currently, there are implementations of approximate computing accelerator designs in PIM based on lookup operations, such as RAPIDNN [25] and NNPIM [19].

### B. Vector Quantization for Approx. Matrix Multiplication

Neural networks inherently encode semantic features of vectors (e.g., a filter in CNN always captures similar input characteristics). Therefore, leveraging these semantic features of vectors in neural networks facilitates approximate model inference. Vector Quantization (VQ, [10], [16], [38]) and Product Quantization (PQ, [13], [14], [29]) are widely used in information coding, information retrieval, similarity search and decompose high-dimensional vectors into (several sets of) lower-dimensional vectors. Recent research on VQ/PQ for Approximate Computing [1], [4], [50], [55] has opened up possibilities to aggressively simplify end-to-end neural network inference.

Fig. 2 illustrates a process to approximate matrix multiplication using VQ. Matrix  $\mathbf{A}_{M,K}$ , representing input data, is segmented into sets of column vectors for each row. Each set undergoes independent clustering via the K-means clustering algorithm to derive  $c$  features (centroids, **step ①**) stored in the set of features (codebooks). During model training, these

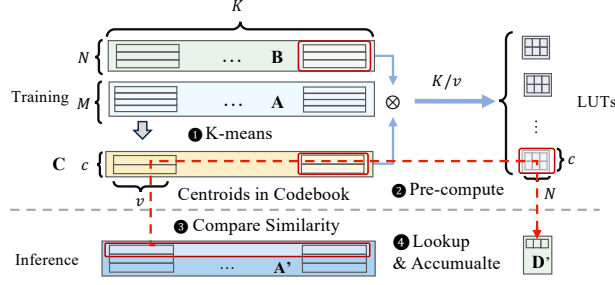


Fig. 2: VQ for Approximating Matrix Multiplication

centroids are updated to minimize loss of model accuracy, while the model weights in the matrix  $\mathbf{B}_{K,N}$  are updated simultaneously.

$$\text{Loss} = \|(\text{LUT}_{N,c}[\text{Index}] \cdot \mathbf{B}_{K,N}) - (\mathbf{A}_{M,K} \cdot \mathbf{B}_{K,N})\|^2$$

We can replace all matrix multiplications in the model with LUTs by performing end-to-end training on the entire model.

For model inference, weights remain constant, allowing pre-computation of multiplication results for all centroids and vectors before inference begins (step ②). It enables efficient matrix multiplication with  $\mathbf{B}_{K,N}$  using the clustered centroids of  $\mathbf{A}_{M,K}$ . When handling an incoming input matrix  $\mathbf{A}'$ , we can evaluate the similarity between input vectors and centroids, then select the most similar features (step ③). The results for the input vector are then retrieved from the precomputed lookup table and accumulated to produce the final matrix multiplication result in  $\mathbf{D}'$  (step ④). Employing VQ significantly reduces computational power and memory requirements, striking a balance between computational efficiency and model accuracy.

### C. Challenges

Although VQ-based AMM effectively improves hardware utilization in AI workload computations and brings new possibilities for extremely low-bit quantization, there are still challenges when applying it in hardware architecture.

**Challenge 1: Hardware Architecture for LUT-based Neural Network is still missing.** Traditional hardware accelerators support computation-intensive tasks like GEMM with Compute-Centric architecture. Previous work has attempted to enhance the performance of LUT-based models. LUT-NN [55] uses CPU vector instructions to speed up lookups, while PIM-DL [32] uses PIM devices to overcome the memory wall. These attempts only enhance execution speed on existing architectures and cannot boost performance by leveraging LUT-based models' unique data patterns and flexible similarity algorithms.

**Challenge 2: LUT-based model conversion is time-consuming and lacks optimization for hardware architectures.** Previous research has shown that converting neural networks to LUT-based networks can be time-consuming and lead to significant accuracy loss [1], [32], [50], [55]. Vector lengths, centroids, and similarity metrics affect LUT-based

model accuracy. Filter settings with excellent accuracy among these vast parameters require a lightweight and fast model conversion algorithm. Furthermore, previous works [1] were unable to use streamlined  $L_1$  similarity to transform models.

**Challenge 3: Interplay of hyper-parameters and extensive search spaces.** In LUT-DLA, the interplay of hyper-parameters significantly influences both model accuracy and hardware design, creating a vast search space that poses a substantial challenge for system optimization. The improved architecture renders traditional CTC-based Design Space Exploration (DSE) schemes [6], [62] inapplicable, necessitating a DSE algorithm designed for the LUT-DLA architecture to explore LUT reuse schemes and parallelism.

## III. LUT-DLA FRAMEWORK OVERVIEW

In this section, we introduce a new co-design framework. Fig. 3 illustrates the architecture of the framework, which consists of:

**LUT-DLA Hardware Generator:** To address **Challenge 1**, we design LUT-DLA Hardware Generator, an agile and parameterized hardware architecture generation framework implemented in Chisel [3]. The hardware architecture generated by the LUT-DLA Hardware Generator primarily consists of Centroid Calculation Modules (CCMs) and In-Memory Matching Modules (IMMs). CCM is designed for similarity comparison, while IMM stores precomputed results and accumulates computation outcomes. LUT-DLA Hardware Generator can quickly generate RTL code based on specified hardware parameters, facilitating further performance evaluation and parameter exploration. Additionally, we introduce a novel, *LUT-Stationary (LS) dataflow*, which effectively reuse on-chip LUTs and helps hide computational delays caused by swapping lookup tables. Sec. IV provides a detailed hardware architecture and dataflow.

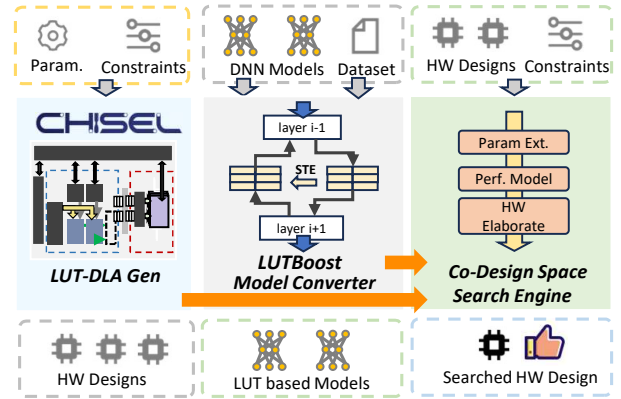


Fig. 3: LUT-DLA Framework

**LUTBoost: Efficient Multi-Stage Model Converter** To address **Challenge 2**, we design a lightweight multistage model training method as Model Converter in Sec. V, which quickly assesses model accuracy and accelerates model convergence. It not only simplifies the design of the model converter but also speeds up training and reduces accuracy loss. In addition,

we integrate more hardware-friendly similarity comparison designs into LUT-based models. These methods significantly reduce hardware area and energy costs while introducing minor errors.

**Co-Design Space Explore Engine:** Both LUT-DLA Hardware Generator and LUTBoost have a large design space. These vast design spaces are prompt us to develop a Co-Design Space Exploration Engine to address **Challenge 3**. As detailed in Sec. VI, we first evaluate the sensitivity of key performance-affecting parameters within the design space. Following this, we select critical parameters and quantitatively model their impact on the system. Building on this foundation, we designed an efficient heuristic search framework with a pruning algorithm to refine the search space.

#### IV. LUT-DLA HARDWARE ARCHITECTURE AND DATAFLOW DESIGN

In this section, we introduce the hardware architecture design of LUT-DLA. Then, we discuss the design choices of dataflows in LUT-DLA and demonstrate how the proposed LUT-Stationary dataflow optimizes the balance between on-chip memory use and memory access.

##### A. Architecture Overview

In Sec. II-B, Fig. 2 introduces the inference of LUT-based neural networks. The execution of each LUT operator is divided into two steps: similarity comparisons and table lookups. Therefore, we partition the hardware architecture of LUT-DLA into two mutually independent parts: **Centroid Computation Modules (CCMs)** and **In-Memory Matching Modules (IMMs)**. Fig. 4 represents hardware architecture. CCMs and IMMs are connected through a group of asynchronous FIFOs, which decouples these modules and allows them to operate in different clock domains.

CCM includes CCUs (**Centroid Computation Units**), centroid buffers, and input buffers. Figure 5 shows that each CCU has several **distance Processing Elements (dPEs)** that can read both centroids and an input vector, do calculations, and compare similarities between the input vector and centroids at the same time. In each computing cycle, CCM loads an input vector into CCU (blue vector in Fig. 5). Each dPE in CCU receives the minimum distance, centroid index, and input vector from the previous dPE, calculates the distance, changes the index, and delivers the input vector to the next dPE to pipeline the similarity comparison.

IMM contains **Indices Buffer**, **PSum LUT (Partial Sum LUT)**, and **Scratchpad**. Every cycle, Indices Buffer retrieves an index from the asynchronous FIFO and obtains the precomputed result from PSum LUT. PSum LUT stores precomputed results of the LUT-based operator, while the scratchpad accumulates and caches the retrieved results.

IMM also supports element-wise activation and dequantization by using polynomial approximations [61]. Other non-matrix multiplication operations using LUTs have been extensively studied in previous work [19], [25] and LUT-DLA can orthogonally employed with the methodologies. For

batch normalization, LUT-DLA could integrate normalization into weights. Regarding operations like softmax/layernorm that require global information, common solutions include offloading these computations to CPU (e.g. NVDLA [45]), Vector Unit (e.g. TPU [28]) or dedicated Special Function Unit [53]. LUT-DLA can assist in runtime sum value collection to accelerate these processes. Since the LUT-DLA architecture only affects the computation path of GEMM, it can achieve a high compatibility with previously optimized methods. For example, if the target network includes pooling operations, we can adding dedicated units on the IMM's data write-back path to collect the values within the pooling window [15], or introducing additional memory access units to handle data pooling [45].

The hardware architecture of flexibly decouples the CCM and IMM designs, enables customized setups for various application scenarios and needs. Our design also supports arbitrary CCM-IMM combinations without timing concerns and workload-based CCM-IMM runtime ratio adaptation. Furthermore, by decoupling CCM and IMM into separate clock domains, LUT-DLA allows the pipeline-designed CCM to run at a higher clock frequency and provide indexes to numerous IMM similarities simultaneously. In the mean time, IMM can operate at a lower clock frequency to reduce power consumption. In Sec. VI-B, we will discuss more details on the mapping of workloads and dataflow of LUT-DLA.

##### B. LUT-DLA Dataflow Exploration

Contemporary hardware accelerators [27], [28], [45], are designed to Compute-Centric to support computation-intensive operations. Dataflows like Weight/Output Stationary are used to increase computational throughput and data reuse. In contrast, LUT-DLA adopts a fundamentally different approach by employing LUTs to accelerate inference, forming a Memory-Centric hardware architecture centered around LUTs. The LUTs serve as both the primary “storage” and “computation” units, leading to performance bottlenecks in the architecture. As LUT-DLA regularly accesses precomputed results from LUTs, a unique dataflow is needed to optimize access, reuse, and minimize hardware costs. We propose **LUT-Stationary (LS) Dataflow** to reduce the size of on-chip look-up tables, reuse data, and preserve acceptable scratchpad memory size.

Consider GEMM operation  $C_{M \times N} = A_{M \times K} \times B_{K \times N}$ , LS traverses the input matrix in a column-major order, first extracting vectors from the same feature space for computation, so the look-up table and centroid matrix can be reused. To evaluate and compare different dataflows, we analyze on-chip memory requirements in GEMM operation. Although a precomputed lookup table must be stored on-chip for  $MNK$ <sup>1</sup>,  $NMK$ , and  $MKN$  computations to decrease repeated loading of the same LUTs,  $KMN$  and  $KNM$  dataflows can cut on-chip space at the expense of a larger scratchpad due to more partial sums in the  $K$  dimension. The  $NKM$  approach

<sup>1</sup>The order of the letters in the table represents the nesting order of the loop when computing  $(M \times K) \times (K \times N)$  GEMM. For example, “MKN” indicates that the innermost loop is N, succeeded externally by K and M.



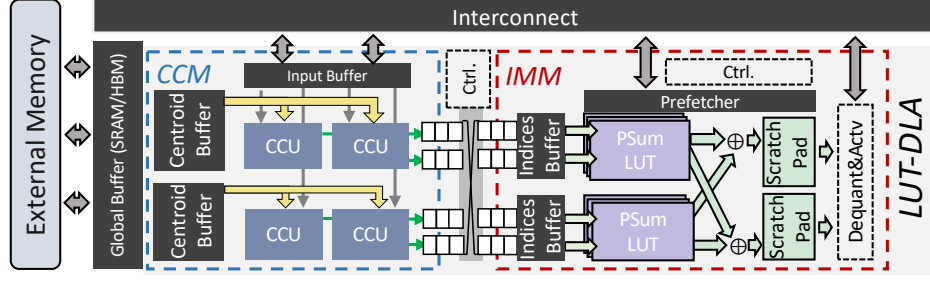


Fig. 4: LUT-DLA Hardware Architecture

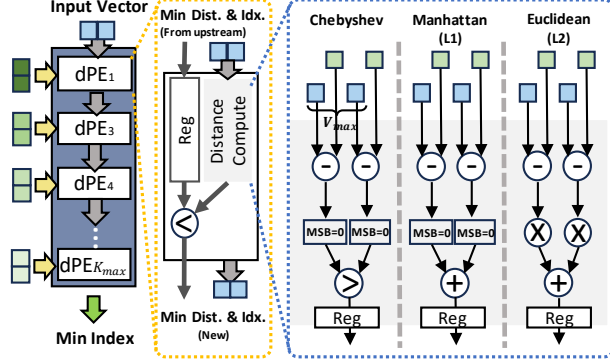


Fig. 5: Centroid Computation Units (CCU) Architecture

adopted by LS dataflow serves as a trade-off between the lookup table and the scratchpad, offering an acceptable size of on-chip storage at the cost of multiple transmissions.

For neural network workloads, the small number of centroids ( $c$ ) enables hardware to hide the LUT loading time using ping-pong buffers while processing prior vectors. Upon finishing traversals on the  $M$  dimension, LS dataflow progresses on the  $K$  dimension to accumulate new partial sums in the scratchpad. This order enables the runtime accumulation of partial sums, thereby minimizing the cost associated with their storage. The traversal of the  $N$  dimension is performed last, using cached indexes in the index buffer to query various sections of the output matrix.

After partitioning into subspaces, we refer to the input matrix as  $A_{M \times (N_c \times v)}$ , where  $N_c = \lceil K/v \rceil$  is the number of partitioned subspaces. As a trade-off between hardware area and performance, we also tile the  $N$  dimension into  $N_o$  parts, where  $N_o = \lceil N/T_n \rceil$ .  $T_n$  is the length of the tile in  $N$  dimension. Algorithm 1 shows the proposed LS dataflow. In previous works [1], [50], I/O overhead caused by large precomputed LUTs has been a major bottleneck. Utilizing LS dataflow facilitates on-demand loading of LUTs, effectively dividing a lengthy and computation-intensive LUT data exchange into several smaller exchanges. While LS does not decrease the overall cost of data transfer, it enhances the utilization of ping-pong buffers for preloading LUTs and employs computation cycles to conceal LUT loading overhead.

Repeatedly loading the same lookup table content will waste storage bandwidth. Given the high cost of reloading the same

Algorithm 1: Pseudocode for LS Dataflow

```

Input : Input Matrix  $A \in \mathbb{R}^{M \times (N_c \times v)}$ ,
        Centroid  $Z \in \mathbb{R}^{c \times (N_c \times v)}$ ,
        precomputed LUT PSum LUT  $\in \mathbb{R}^{c \times N_c \times N_o \times T_n}$ 
Output: Output Matrix  $C \in \mathbb{R}^{M \times (N_o \times T_n)}$ 
1 for  $n \leftarrow 0$  to  $N_o$  do
2   for  $k \leftarrow 0$  to  $N_c$  do
3     PSum LUT[...]  $\leftarrow$  PSum LUT[...][k][n];
4     for  $m \leftarrow 0$  to  $M$  do
5       if  $k=0$  then
6         // CCM: Compare similarity and
           get index
7         idx = GetIndex( $A[m][k]$ ,  $Z[...][k]$ );
8          $f.push(idx)$ ;
9         IndicesBuffer[m] =  $f.front()$ ;
10      end
11      // IMM: Query and Accumulate
12      idx = IndicesBuffer[m];
13      PSum Buffer[m] += PSum LUT[idx];
14    end
15  end
16 end

```

LUTs, we calculated the resource requirements for different dataflows, as shown in Table I. The results listed in the table are the minimum sizes that *does not load same LUT more than once*. Dataflow column shows the specifies the loop order.

TABLE I: Comparison of Dataflow Impact on On-chip Memory ( $M = 512$ ,  $K = N = 768$ ,  $v = 4$ ,  $c = 32$ )

| Dataflow <sup>1</sup> | Scratchpad | Indices Buffer | PSumLUT | Total Size |
|-----------------------|------------|----------------|---------|------------|
| MNK                   | 0.03KB     | 0.05KB         | 2064KB  | 2064.1KB   |
| NMK                   | 0.03KB     | 26.9KB         | 2064KB  | 2090.9KB   |
| MKN                   | 0.75KB     | 0.6B           | 2064KB  | 2064.8KB   |
| KMN                   | 384KB      | 0.6B           | 24KB    | 408.0KB    |
| KNM                   | 384KB      | 0.31KB         | 1KB     | 385.3KB    |
| LUT-Stationary        | 16KB       | 0.31KB         | 1KB     | 17.3KB     |

<sup>1</sup>  $M$ ,  $K$ , and  $N$  indicate the order of the for-loops from outer to inner.

## V. LUTBOOST: EFFICIENT MODEL CONVERTER

LUT-based models extract semantic vectors from input data during training and retain precomputed results in lookup tables for inference. There are typically two methods to obtain LUT-based models: **First**, training a model from scratch [1], [4], [50], which involves initially randomizing LUTs and model

weights and then retraining it according to the task. This approach guarantees high accuracy but requires extensive GPU hours, presenting considerable difficulties in converting larger models. **Second**, transforming well-trained models into LUT-based models [32], by generating LUTs from original model weights and performing minor finetuning to recover accuracy, can significantly reduce GPU training time. However, the transformation of LUT-based models is non-differentiable and unstable; such instability notably impacts model convergence and accuracy.

This section introduces LUTBoost, a novel, lightweight, and efficient LUT-based model transformation algorithm. LUTBoost can convert LUT-DLA models and fine-tune any LUT-based models independently. This algorithm outperforms previous work in convergence speed and accuracy [1], [32], [50], [55], while supporting more efficient similarity computation operators. To achieve these, LUTBoost designs a multistage training strategy to enhance model convergence and avoid training from scratch and proposes a new LUT reconstruction strategy that improves the loss function, making model training more stable across similarity computation processes.

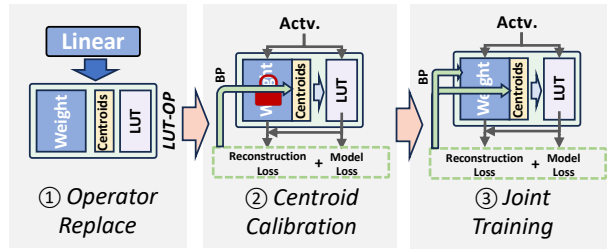


Fig. 6: LUTBoost: Lightweight Multistage Model Converter

1) *Efficient Multistage Model Transformation*: LUTBoost simplifies transformation, speeds up training, and reduces accuracy loss by dividing it into three concentrated steps. Previous works [1], [50], [55] train centroids and weights concurrently, rather than in stages, resulting in weights overfitting to initially suboptimal centroids. For example, ResNet18 has 11.69M parameters, but the LUT-based model has only 542.29K centroids (a 4% difference from weights). These parameters are crucial because they represent input data for each network layer. If these centroids misrepresent feature semantic similarity, the model's accuracy can suffer despite their small number. Previous single-stage techniques randomly initialize centroids, disturbing the well-trained original weight distributions and easily causing weights to converge on features represented by untrained centroids. In LUTBoost, we first train the centroids to improve representation, then train both centroids and weights to avoid errors caused by poorly trained centroids.

Fig. 6 illustrates the progressive training pipeline. In step ①, LUTBoost substitutes traditional linear operators (matrix multiplication) with more efficient LUT operators. Then, LUTBoost freezes all parameters except the centroids and trains all centroids on the dataset in step ②. After calibrating the

centroids, we train both centroids and weights in step ③ to restore and improve accuracy. A significant advantage of multistage model training is its ability to quickly evaluate model accuracy under current parameter configurations within a short period. It offers an agile estimation for algorithm and hardware co-design space exploration, enabling efficient identification of optimal configurations in Sec. VI-B.

Fig. 7 shows the progress of BERT training using LUTBoost. Compared to single-stage training [32] (BERT-base, vector=4, centroid=64), the multistage method significantly decreases training loss by centroids in 2000 iterations and achieves faster and better convergence by training both centroids and weights together.

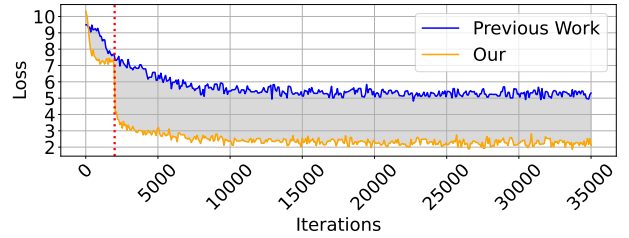


Fig. 7: Multistage Model Training Comparison

2) *Hardware-Friendly Feature Similarity Comparison*: In LUT-based models, feature similarity comparison is computationally intensive and challenges hardware cost optimization. Traditionally, the most similar feature has been selected using *Euclidean Distance* ( $L_2$  distance) between vectors and centroids as:

$$\text{similarity} = \sum \|\mathbf{V} - \mathbf{C}\|^2$$

$L_2$  distance is differentiable and facilitates efficient training with smooth, continuous gradients. However, it is hardware resource-intensive, which requires enormous floating-point multipliers and adders.

LUTBoost expands the design space for similarity comparison by additionally supporting *Manhattan Distance* ( $L_1$  distance) and *Chebyshev Distance*. It allows for a better balance between model accuracy and hardware costs. *Manhattan Distance* ( $L_1$  distance) sums the absolute differences of vectors, which is computationally easier than  $L_2$  since it involves only addition and absolute operators, without multiplication:

$$\text{similarity} = \sum |\mathbf{V} - \mathbf{C}|$$

$L_1$  distance is more hardware-efficient, particularly when comparing multiple centroids. *Chebyshev Distance* is a much more aggressive approach to measuring distance, defined as the maximum of absolute differences along any coordinate dimension:

$$\text{similarity} = \max(|\mathbf{V} - \mathbf{C}|)$$

It only involves evaluating absolute differences followed by identifying the maximum value, which is generally less computationally intensive than  $L_1$  distance, making Chebyshev distance more hardware-efficient than  $L_1$ .

The  $L_1$  and Chebyshev distances reduce computational complexity and hardware costs but introduce non-differentiable operations that hinder model convergence stability. We employed a *Straight-Through Estimator (STE)* to estimate gradients in back-propagation by treating specific non-differentiable functions as differentiable. STE employs the derivative of a related function in the backward pass, while the forward pass remains unchanged. For example, for a non-differentiable function with a discontinuity at zero, STE might approximate its derivative as a constant in the backward pass using the sign function during the forward pass. The approximation enables numerical gradient computation by retaining forward function values and employing alternate gradient values in backpropagation.

In LUTBoost, we introduce a reconstruction loss (Eq. (V-2), where  $\text{SG}(\cdot)$  denotes the stop gradient operator) to improve centroid updates, typically expressed using a uniform equation. It serves as a regularization loss function, aligning centroids with the original activation or input data for a high-quality representation. Minimizing this loss decreases error accumulation from approximating non-differentiable components during training, improving centroid representation fidelity and model training robustness.

$$L_{\text{re}} = (\text{SG}(\hat{A} \cdot W) - A \cdot W)^2 + (\hat{A} \cdot W - \text{SG}(A \cdot W))^2$$

It treats input data ( $\hat{A}$ ) as centroids to propagate errors introduced by centroids.

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \hat{A}} \underbrace{\frac{\partial \hat{A}}{\partial A}}_{\text{straight through}} \approx \frac{\partial L}{\partial \hat{A}}$$

During the backward pass, it utilizes the original activations or input data ( $A$ ) to circumvent non-differentiable components.

$$\text{output} = \begin{cases} \hat{A} \cdot W, & \text{forward propagation} \\ A \cdot W, & \text{backward propagation} \end{cases}$$

Building on these optimizations, LUTBoost achieves the training approach that not only rapidly converges and achieves high accuracy but also balances the complexity and cost of hardware architecture design. Table II presents examples where we train ResNet20/32/56 models on the CIFAR-100 dataset, with adjustments to training strategies and similarity. The results show that multi-stage training can enhance model accuracy by 3.27-5.84% in  $L_2$  and 5.57-7.20% in  $L_1$ . Compared to  $L_2$ ,  $L_1$  slightly reduces model accuracy (Multistage  $L_2 - L_1$ , 0.25-0.46%), but significantly simplifies hardware architecture design and expands the dimensions to explore hardware design space (will be discussed in Sec. VII-B).

## VI. CO-DESIGN SPACE SEARCH ENGINE

LUT-DLA offers a broad design space for various applications, encompassing numerous suboptimal design points. In this section, we first explore Design Space Exploration (DSE) in hardware architecture and LUTBoost. Then, we model the search space using prior studies and propose an

TABLE II: LUTBoost Training Evaluation

| Model    | Single Stage |       | Multi Stage   |               |
|----------|--------------|-------|---------------|---------------|
|          | $L_2$        | $L_1$ | $L_2$         | $L_1$         |
| ResNet20 | 60.37        | 60.06 | 66.21 (+5.84) | 65.91 (+5.85) |
| ResNet32 | 64.61        | 62.06 | 67.88 (+3.27) | 67.63 (+5.57) |
| ResNet56 | 65.00        | 62.58 | 70.24 (+5.24) | 69.78 (+7.20) |

efficient co-design search algorithm based on this modeling. The DSE algorithm faces two challenges: **First**, the DSE search involves shared parameters ( $v$ ,  $c$ ) across software and hardware, complicating optimization. **Second**, DSE requires a thorough analysis of dataflow and decoupled architecture to enhance computation unit utilization.

### A. Design Space Exploration

1) **Model Accuracy Sensitivity:** LUTBoost relies on three critical parameters: vector lengths, the number of centroids, and similarity metrics.

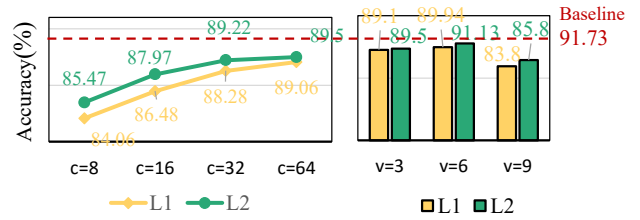


Fig. 8: Sensitivity Analysis of ResNet20 on CIFAR10 (Left: Number of centroids; Right: vector length)

**Vector length:** Vector length affects the compression ratio of LUT-NN; longer vectors increase both the compression ratio and accuracy drop. In Fig. 8, we evaluate ResNet20 performance at vector lengths with a constant number of centroids. Results show that shorter vector lengths ( $v = 3$ ) achieve higher accuracy than longer ones ( $v = 9$ ). In our experiments, nearly all models showed an enhancement in accuracy<sup>2</sup> as  $v$  decreased. This trend is due to high-dimensional vectors posing challenges in extracting effective similarity measures, which makes learning the structure of features difficult. A reduced  $v$  also indicates more subspaces, introducing additional trainable parameters.

**Number of centroids:** The number of centroids determine the capability in vector spaces. As shown in Fig. 8, while more centroids generally enhance peak accuracy, the benefits diminish as the numbers increase. Specifically, 32 centroids often have already captured essential patterns effectively; additional centroids contribute only marginal improvements. It indicates an optimal centroid range that effectively balances model complexity and expressive capability.

**Similarity metrics:** LUT-DLA supports various similarity metrics to measure similarity of features, which can significantly influence model accuracy. Fig. 8 shows the impact of different distance metrics on model performance. Specifically,

<sup>2</sup>This accuracy is defined as the accuracy result when the loss no longer decreases within 100 training epochs.

switching from  $L_2$  distance to the lightweight  $L_1$  distance results in a slight accuracy drop. However, the slight accuracy drop can significantly save hardware resources, reducing both area and energy costs in Sec. VI-A2.

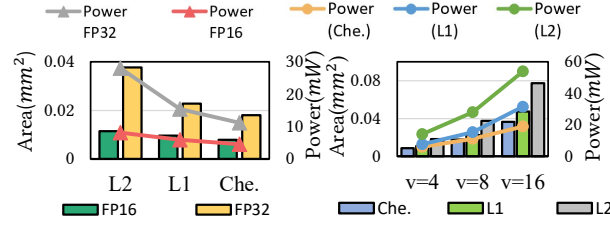


Fig. 9: **Left:** Area and Energy Overhead of dPE when  $v = 8$ . **Right:** Hardware Overhead Under Different  $v$

2) **Hardware Resource Sensitivity:** The main factors affecting the computational component are vector length, similarity metrics, and numeric precision.

**Vector length:** Vector length impacts the number of element-wise operations in similarity computations and the size of the reduction tree in hardware. As shown in Fig. 9, area and power consumption increase approximately linearly with vector length. However, due to the non-linear scaling costs of reduction trees in hardware, the increase in consumption is not directly proportional to the increase in vector length.

**Similarity metrics:** We also examine the impact of different feature similarity comparison methods on hardware in Fig. 9.  $L_1$  distance replaces hardware multiplier in the  $L_2$  distance, and reduces the area and power consumption. This improvement is not affected by the length of the vector. Chebyshev distance further reduces hardware costs and increases hardware efficiency, which benefits LUT-DLA deployment in scenarios requiring extremely low cost and power.

**Numeric precision:** Although LUTBoost and traditional quantization optimizations are orthogonal approaches, by adopting lower-precision floating-point representations for similarity calculations, we can significantly reduce both the computational overhead and the chip area required for similarity comparison. Fig. 9 shows the chip area and power consumption with different precision.

### B. Quantitative Search Space Modeling

After evaluating the sensitivity of the parameters, we are able to quantitatively model between hyper-parameters and software/hardware costs based on the algorithm and architectural characteristics of LUT-DLA.

1) **Computational Modeling:** In Fig. 2, LUT-based approximation of matrix multiplication utilizes vector quantization to represent input data by similarity comparison. The computational cost-utility function  $\tau$  can be modeled by Eq. (1). The function includes computations for similarity comparisons and accumulation. The constant  $\alpha_{\text{sim}}$  represents the computational cost needed for similarity calculation; for example, for  $L_2$  distance,  $\alpha_{\text{sim}} = 2$  accounts for 1 multiplier and 1 adder.

TABLE III: LUT-DLA Modeling Symbols

| Symbol           | Description                                                                                      |
|------------------|--------------------------------------------------------------------------------------------------|
| $M/K/N$          | The size of the matrix used for modeling (Input feature: $M \times K$ and Weight: $K \times N$ ) |
| $v$              | Vector length                                                                                    |
| $c$              | Number of centroids in a codebook                                                                |
| $\beta$          | Memory bandwidth (bit/cycle)                                                                     |
| $n_{\text{CCU}}$ | Number of CCU in CCM                                                                             |
| $n_{\text{IMM}}$ | Number of IMM                                                                                    |

The utility function  $\tau$  determine the acceleration ratio obtained from computational savings.

$$\tau(v, c) = \text{OP}_{\text{sim}} + \text{OP}_{\text{add}} = \alpha_{\text{sim}} c M \phi \left[ \frac{c}{\phi} \right] + MN \left[ \frac{K}{v} \right] \quad (1)$$

2) **Memory Access Modeling:**  $\phi$  describes memory size. Memory cost in LUT-DLA comprises three parts: input, scratchpad, and LUT memory. LUT-DLA requires additional memory to store the lookup tables. It is also a key trade-off to balance LUTs with computation reduction.

$$\begin{aligned} \phi(v, c) &= \text{mem}_{\text{in}} + \text{mem}_{\text{out}} + \text{mem}_{\text{LUT}} \\ &= Nc \frac{K}{v} \text{bit}_{\text{lut}} + MN \text{bit}_{\text{out}} + \frac{K}{v} M \log_2 c \end{aligned} \quad (2)$$

3) **Hardware Cost Modeling:** Hardware resources can be estimated using modeling function  $\varphi_{\text{area}}$  and  $\varphi_{\text{power}}$ . We can break down hardware costs in CCU and IMM in Eq. (3) and Eq. (4). IMM primarily comprises memory blocks, which can rapidly generate and estimate by memory compilers. The main hardware costs of CCU are PE Array and Centroid Buffer. PE Array and adders in IMM costs can be approximated based on the vector length and the similarity comparison method using standard arithmetic libraries.

$$\begin{aligned} \varphi_{\text{area}}(v, c, n_{\text{IMM}}, n_{\text{CCU}}) \\ = \text{area}_{\text{IMM}} * n_{\text{IMM}} + \text{area}_{\text{CCU}} * n_{\text{CCU}} + \text{area}_{\text{other}} \end{aligned} \quad (3)$$

$$\begin{aligned} \varphi_{\text{power}}(v, c, n_{\text{IMM}}, n_{\text{CCU}}) \\ = \text{power}_{\text{IMM}} * n_{\text{IMM}} + \text{power}_{\text{CCU}} * n_{\text{CCU}} + \text{power}_{\text{other}} \end{aligned} \quad (4)$$

4) **Parallelism Modeling:** Due to variations in model sizes and algorithm configurations, LUT-based models vary in similarity computations, LUT capacity, and memory access. LUT-DLA uses multiple CCUs and IMM to facilitate parallel processing of these tasks, enhancing hardware efficiency and model accuracy. We quantitatively assess the impact of varying numbers of CCUs and IMM on system performance. As shown in Fig. 10, a LUT operator performs three crucial steps: similarity comparison, LUT loading, and table lookup. LUT loading moves precomputed results from off-chip to on-chip memory, and similarity computation evaluates input vector and centroid similarities concurrently. The table lookup then accesses these results from on-chip memory. Eq. (5) quantifies the clock cycles required for these steps.

$$\begin{aligned} \omega(v, c, \beta, n_{\text{IMM}}, n_{\text{CCU}}) &= \max(\text{load}, \text{sim}, \text{lut}) \\ &= \max\left(\frac{c \text{bit}_{\text{LUT}}}{\beta} n_{\text{IMM}}, \frac{MK}{vn_{\text{CCU}}}, \frac{MNK}{vn_{\text{IMM}}}\right) \end{aligned} \quad (5)$$



Since these three steps are sequentially dependent, we align these phases to the maximum clock cycles among them as the measurement metric. It aims to ensure similar cycles across these steps for pipeline stage balancing. Fig. 10 shows an example in which doubling the number of IMMs in the current scenario can double the system's throughput by enabling hardware reuse for similarity computations.

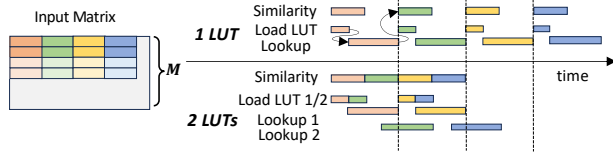


Fig. 10: Expanding the Lookup-Limited Design to Increase the Utilization of The Compute Array and Boost Throughput

### C. Co-Design Space Search Engine

The Co-Design space search engine explores the entire design space. The objective is to minimize the maximum cost that significantly affects the hardware architecture. To address the challenges proposed in Sec. VI, we first use the analytical model to trim the search space and utilize the parameter sensitivity of model accuracy analyzed in Sec. VI-A1 for rapid accuracy search. Finally, we employ a greedy strategy to improve the utilization of computational resources and determine the optimal design.

$$\begin{aligned} \min \quad & \omega(v, c, \beta, n_{\text{IMM}}, n_{\text{CCU}}) \\ \text{s.t.} \quad & \tau, \phi \leq \text{GEMM Requirements} \\ & \varphi_{\text{area}}, \varphi_{\text{power}} \leq \text{HW Constraints} \\ & \text{LUTBoost}(v, c) \geq \text{Accuracy constraints} \end{aligned}$$

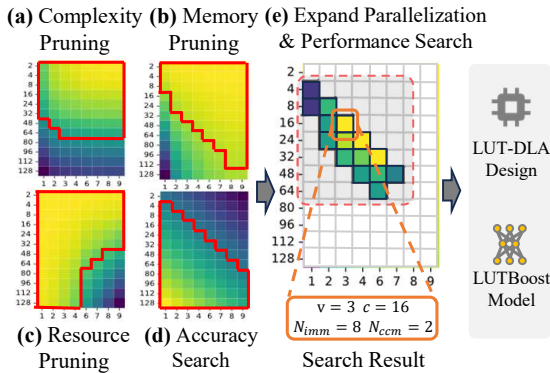


Fig. 11: Co-Design Space Search Engine Example

Algorithm 2 represents our heuristic search. Fig. 11 illustrates an example of finding a suitable LUT-DLA configuration for ResNet series models. In this figure, the design space for each step is depicted using a 2-D heatmap, with the horizontal axis representing sub-vector length, the vertical axis representing the number of centroids, and the color indicating the data results produced by hardware modeling. Initially, we employ a software model to rapidly identify design points that meet the constraints (Fig. 11 a,b,c and d), marking them

### Algorithm 2: Co-Design Space Search Engine Algorithm

---

**Input :** Search Space  $S = V \times C$ ,  
power/performance/area/accuracy constraints

**Output:**  $v, c, n_{\text{CCU}}, n_{\text{IMM}}$

```

1 Prune search space  $S'$  by Eq.(1), Eq.(2) // Step 1
2 for  $(v, c)$  in  $S'$  do
3   if Eq.(5)  $\leq$  accuracy constraint // Step 2
4   then
5      $n_{\text{IMM}} \leftarrow 1; n_{\text{CCU}} \leftarrow 1$ 
6     while Eq.(3)  $\leq$  area constraint & // Step 3
7       Eq.(4)  $\leq$  power constraint) do
8       if  $n_{\text{IMM}} < n_{\text{CCU}} \cdot N$  // Step 4
9       then
10         $n_{\text{IMM}} \leftarrow n_{\text{IMM}} + 1$  // IMM-Bound
11      end
12    else
13       $n_{\text{CCU}} \leftarrow n_{\text{CCU}} + 1$  // CCU-Bound
14    end
15  end
16 end
17 end

```

---

with red boxes. The intersection of these points is then used as the final search set for detailed end-to-end performance simulations (Fig. 11 e).

**Step 1: Pruning by Computation and Memory:** Fig. 11 (a) and (b) portions of the design space using computation and memory access model, Exceed the computation and memory constraints, and search space are pruned, resulting in  $S'$ . It allows us to eliminate bad configurations that lead to worse results than traditional GEMMs.

**Step 2: Pruning by Hardware Constraints:** Fig. 11 (c) further utilizes hardware models to exclude out-of-constraint designs.

**Step 3: Heuristic Search for Coarse-Grained Accuracy:** As discussed in Sec. V, LUTBoost can rapidly estimate the model's accuracy in the early training stage with minimal costs. We employ a coarse-grained accuracy search to exclude design configurations that demonstrate low accuracy. It allows us to quickly narrow the design space, as in Fig. 11 (d).

**Step 4: Parallelism Expansion,** Among the remaining design configurations, we will combine the performance model and resource model, explore parallelism of design, and search for the most efficient design that meets the area constraints. Eq.(5) shows as input matrix shape increases (commonly after im2col), the throughput of the LUT data load is not limited, but similarity computation will be the bottleneck. In previous work [6], [62], DSE often adapts the size of the PE array to the model's compute pattern through the Computation to Communication (CTC) ratio, which can be calculated according to the roofline model due to bandwidth limitations.

LUT-DLA completely decouples the lookup and computation phases, allowing idle CCUs to perform similarity computations for other IMMs (Figure 10). When table lookup becomes the main limiting factor, the DSE algorithm will increase CCU utilization by inserting additional IMMs. To

TABLE IV: Accuracy of LUT-Based Models

|                 |               | FP32+FP32 |       | BF16+INT8 |       | BaseLine |
|-----------------|---------------|-----------|-------|-----------|-------|----------|
|                 |               | L2        | L1    | L2        | L1    | FP32     |
| <b>ResNet20</b> | CIFAR10       | 91.19     | 90.06 | 90.76     | 89.63 | 91.73    |
|                 | CIFAR100      | 66.21     | 65.91 | 65.8      | 65.66 | 68.83    |
| <b>ResNet32</b> | CIFAR10       | 92.10     | 91.33 | 91.68     | 91.18 | 92.63    |
|                 | CIFAR100      | 67.88     | 67.63 | 67.06     | 66.92 | 70.16    |
| <b>ResNet56</b> | CIFAR10       | 92.22     | 91.67 | 91.65     | 91.53 | 93.39    |
|                 | CIFAR100      | 70.24     | 69.78 | 69.7      | 69.27 | 72.63    |
| <b>ResNet18</b> | CIFAR100      | 75.19     | 75.23 | 74.66     | 74.61 | 77.19    |
|                 | Tiny-Imagenet | 58.67     | 58.46 | 58.33     | 58.27 | 59.19    |
| <b>VGG11</b>    | Imagenet      | 67.11     | 67.65 | 66.79     | 66.08 | 69.40    |
|                 | CIFAR10       | 93.59     | 93.21 | 93.29     | 93.09 | 94.81    |
| <b>LeNet</b>    | CIFAR100      | 72.54     | 72.14 | 72.16     | 71.58 | 74.95    |
|                 | MNIST         | 99.35     | 99.14 | 99.32     | 99.07 | 99.38    |

TABLE V: Bitwidth and Similarity Evaluation (ResNet20)

| Equivalent Bit  |     | 0.3bit | 0.4bit | 0.5bit | 0.7bit | 1bit  | 1.3bit |
|-----------------|-----|--------|--------|--------|--------|-------|--------|
| <b>Params</b>   | $v$ | 9      | 9      | 6      | 6      | 3     | 3      |
|                 | $c$ | 8      | 16     | 8      | 16     | 8     | 16     |
| <b>Accuracy</b> | L2  | 87.78  | 89.45  | 89.18  | 90.18  | 90.48 | 90.78  |
|                 | L1  | 87.18  | 88.47  | 87.58  | 88.53  | 89.08 | 89.48  |

fully exploit architecture, we apply a LUT-first greedy strategy to expand parallelism to save area according to model requirements. Fig. 11 (e) shows the strategy, which improves the theoretical performance of the final design to meet constraints.

In the end, we execute RTL synthesis in descending order of theoretical performance and complete training with the selected parameters.

## VII. EXPERIMENT AND EVALUATION

### A. Model Accuracy

**Settings** For ResNet models, we align with previous [55] setup, and we first freeze weights and perform centroid learning for 20 epochs with learning rate:  $lr = 1 \times 10^{-3}$ , then we jointly train weights and centroids with  $lr = 5 \times 10^{-4}$  for 300 epochs. We set a penalty ratio of 0.05 for reconstruction loss. For BERT/DistilBERT, we employ  $lr = 1 \times 10^{-3}$  for centroid learning for 2000 iterations, with penalty  $1 \times 10^{-2}$  for the reconstruction. In the joint training stage, the learning rate is  $5 \times 10^{-5}$ , and training cost for 190K /390K iterations, with penalty  $1 \times 10^{-1}$ . For OPT-125M, we use  $lr = 1 \times 10^{-3}$  and train 3 epochs to achieve centroid convergence. Subsequently, we conduct joint training for 10 epochs with  $lr = 5 \times 10^{-5}$ . The penalty of reconstruction loss is set to 0.1 in the centroid training stage and  $1 \times 10^{-2}$  in the joint training stage.

We also compare our method with PECAN [50] and PQA [1] using their paper’s original settings under the same sub-vector length ( $v$ ) and number of centroids ( $c$ ).

**LUTBoost Model Accuracy** In contrast to earlier LUT-based training algorithms [1], [50], [55], LUTBoost can train deep neural networks with sufficient accuracy on huge datasets. Table IV shows the accuracy of common CNN models. LUTBoost is nearly as accurate as the baseline on these models, with an average accuracy reduction of 1.2%/2.9% on CIFAR10/100. We also conducted quantization studies,

and experiments show that traditional quantization methods are orthogonal to LUTBoost. BF16 distance computation and INT8 lookup table reduce accuracy by  $< 1\%$  while reducing on-chip area overhead and data moving costs by  $4\times$ .

To accommodate larger and more complex models and datasets, we further test ResNet18 along with Tiny-Imagenet and Imagenet. With a multi-stage training technique, LUTBoost achieved remarkable model convergence and accuracy, with only 0.8%~2.6% loss on these datasets. Table VI compares BERT, DistilBERT, and OPT-125M on GLUE [58] with various similarity computation techniques. While LUT-NN [55] results in a drastic accuracy drop in these tasks, LUTBoost maintains a competitive edge, with an average accuracy consistently above 84% on these tasks in the GLUE benchmark. Our technique also outperforms the state-of-the-art training algorithm eLUT-NN in the GLUE dataset, while maintaining competitive accuracy on sub-tasks, proving its robustness. Our investigations revealed that larger models demonstrate superior fault tolerance, as ResNet18 (11.7M parameters) retains accuracy under more aggressive quantization parameters than ResNet20 (270K parameters). We believe that LUTBoost can efficiently scale to LLMs or large CNN networks. As an experiment, we used LUTBoost to train OPT-125M. To the best of our knowledge, this is the first time a LUT-based model has been scaled to such a size, underlining its potential for widespread application in various transformer-based models.

**Compare with Baseline** The quantization error of LUT-based models follows the same pattern as traditional models. For larger models (ResNet18) on simpler datasets (CIFAR10), the quantization loss using similarity distance for inference is only about 0.1%. However, when it comes to larger datasets like CIFAR100, which are inherently more difficult to converge during training, there is a relatively noticeable quantization loss. The maximum accuracy drop is observed as 3.1% (ResNet32 using  $L_2$ ), 3.4% (ResNet56 using  $L_1$ ), and 3.8% (ResNet56 using Chebyshev). As for Transformer-based models, employing  $L_2$  similarity on DistilBERT results in minimal quantization loss, approximately 1.4%. In contrast, utilizing  $L_1$  similarity on BERT can lead to a loss of up to 3.0%.

**Compare between  $L_1$  and  $L_2$**  In Table IV, the model using  $L_1$  distance incurs only an accuracy drop  $\sim 1\%$  lower than that of  $L_2$ . This eliminates the large loss from Manhattan distance, as indicated in [1], enabling multiplication-free neural network inference. The performance of Chebyshev distance is also comparable, demonstrating a similar accuracy drop as the  $L_1$  distance, which implies a robust alternative to traditional distances.

**Bitwidth and Similarity Evaluation** Table V provides an in-depth presentation of the accuracy results of ResNet20 under different equivalent bit widths. During inference, a subvector of the input matrix is represented by the centroid index within its subspace; the equivalent bit can be computed as  $\lceil \log_2 c \rceil / v$ . As described in Sec. VI-A1, an increase in  $c$  and a decrease in  $v$  can enhance the equivalent bit width, thus

TABLE VI: LUT-based Transformer Model Accuracy on GLUE

| Method     | Setting        | Model      | SST-2          | QQP            | QNLI           | MNLI           | MRPC           | STS-B          | Average        |
|------------|----------------|------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| LUT-NN     | first 6 layers | BERT       | 49.3           | 63.2           | 50.6           | 35.5           | 31.6           | 1.36           | 38.6           |
|            | LUT-based only |            |                |                |                |                |                |                |                |
| eLUT-NN    | $L_2$ only     |            | 92.4           | 69.6           | 87.4           | 79.9           | 87.1           | 83.2           | 76.9           |
|            | Baseline/      |            | 91.9/89.1/89.8 | 90.4/88.4/89.1 | 90.0/85.7/85.7 | 82.3/78.6/79.1 | 84.8/81.9/81.9 | 87.1/84.3/84.7 | 87.7/84.7/85.1 |
| <b>Our</b> | $L_1$ /        | OPT-125M   | 92.7/90.5/91.4 | 90.2/88.4/89.2 | 88.7/86.5/87.1 | 82.1/79.1/79.5 | 83.5/80.4/80.6 | 86.1/84.4/84.4 | 87.2/84.9/85.4 |
|            | $L_2$          | DistilBERT | 90.3/88.9/89.4 | 89.5/88.1/88.8 | 88.1/85.0/85.6 | 80.8/78.1/78.8 | 83.8/80.9/82.4 | 86.1/83.6/84.6 | 86.4/84.1/85.0 |

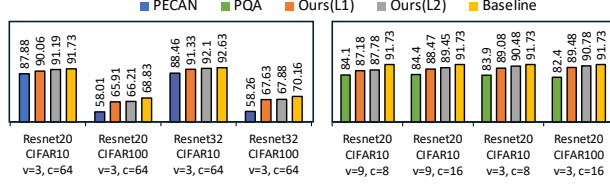


Fig. 12: Comparison with PECAN and PQA

TABLE VII: IMM Settings and Resources Need

|                | V | N <sub>c</sub> | T <sub>n</sub> | M   | SRAM    | Bandwidth |
|----------------|---|----------------|----------------|-----|---------|-----------|
| <b>Design1</b> | 3 | 16             | 128            | 256 | 36.1KB  | 4.1GB/s   |
| <b>Design2</b> | 4 | 16             | 256            | 256 | 72.1KB  | 7.0GB/s   |
| <b>Design3</b> | 3 | 16             | 768            | 512 | 408.2KB | 8.7GB/s   |

facilitating feature extraction. However, simultaneous changes to  $c$  and  $v$  can sometimes lead to unpredictable results. For example, when the equivalent bit width is 0.5, its accuracy decreases a little bit compared to the 0.4-bit model. It because the accuracy of LUT-based models is also dependent on their inherent data distribution. Reducing the subvector length may result in outliers in some subspaces exerting a greater influence on clustering, which may be challenging to mitigate merely by augmenting the centroids number.

**Comparison with Previous Works** Compared with PECAN and PQA, due to the difficulty in convergence of their training algorithm, they only investigated the accuracy of small models on simple datasets. Since the their end-to-end training algorithm is not open-sourced, we can only align parameter settings using our own training algorithm. Figure 12 shows a significant improvement using our method. LUTBoost outperforms PECAN with an average accuracy increase of 2.5% on CIFAR10 and 8.2% on CIFAR100. Compared to PQA, LUTBoost also achieves a notable improvement, ranging from approximately 3.7% to 8.4% in various settings.

### B. Power, Performance and Area Analysis

**Settings** To demonstrate the effectiveness of LUT-DLA and the co-design engine, we generate three LUT-DLA designs under different constraints using our DSE algorithm. **Design 1 (Tiny)** is similar to NVDLA-Small, the smallest area among all benchmark designs. **Design 2 (Large)** has similar throughput to NVDLA-Large, the highest throughput benchmark design. Design 2 can also demonstrate the architectural advantages of LUT-DLA in similar power consumption situations (compared to Gemmini). **Design 3 (Fit)** is the most efficient architecture searched by the co-design engine. We implement these designs using Chisel HDL [3]. Memory (Reg-

TABLE VIII: Comparison with Other Accelerators

|                                | Tech. | Freq | Area               | Power   | Perf.   | Area                                      | Power                       |                   |
|--------------------------------|-------|------|--------------------|---------|---------|-------------------------------------------|-----------------------------|-------------------|
|                                | (nm)  | (M)  | (mm <sup>2</sup> ) | (mW)    | (GOPS)  | Eff. (GOPS/mm <sup>2</sup> ) <sup>a</sup> | Eff. (GOPS/mW) <sup>a</sup> | Func <sup>b</sup> |
| <b>NVIDIA A100 [9]</b>         | 7     | 1512 | 826                | 300000  | 624000  | 18.6                                      | 0.2                         | C/T               |
| <b>Gemmini [15]</b>            | 16    | 500  | 1.21               | 312.41  | 256     | 86.7                                      | 0.8                         | C/T               |
| <b>NVDLA Small/Large [45]</b>  | 28    | 1000 | 0.91/5.5           | 55/766  | 64/2048 | 70.3/372.4                                | 1.2/2.7                     | C                 |
| <b>ELSA [20]</b>               | 40    | 1000 | 2.147              | 1047.08 | 1088    | 508.4                                     | 1.0                         | T                 |
| <b>FACT [49]</b>               | 28    | 500  | 6.03               | 337.07  | 928     | 153.9                                     | 2.8                         | T                 |
| <b>RRAM-DNN [34]</b>           | 22    | 120  | 10.8               | 127.9   | 123     | 5.2                                       | 0.9                         | C                 |
| <b>LUT-DLA Design1 (Tiny)</b>  | 28    | 300  | 0.755              | 219.57  | 460.8   | 610.3                                     | 2.1                         | C/T               |
| <b>LUT-DLA Design2 (Large)</b> | 28    | 300  | 1.701              | 314.975 | 1228.8  | 722.3                                     | 3.9                         | C/T               |
| <b>LUT-DLA Design3 (Fit)</b>   | 28    | 300  | 3.64               | 496.4   | 2764.8  | 759.5                                     | 5.6                         | C/T               |

<sup>a</sup> The energy and area efficiency are scaled to the same process node by scaling [54].

<sup>b</sup> Func Abbr.: C:CNN, T:Transformer

TABLE IX: Comparison with LUT-Based DLA

|                | On-Chip Mem (KB) <sup>a</sup> | Cycles (k) <sup>b</sup> | Dataflow | Pipelined | PingPong Buffer | Large Models |
|----------------|-------------------------------|-------------------------|----------|-----------|-----------------|--------------|
| <b>PQA [1]</b> | 6912.25                       | 7864                    | -        | ✓         | ✗               | ✗            |
| <b>LUT-DLA</b> | 10.5                          | 4743                    | LS       | ✓         | ✓               | ✓            |

<sup>a</sup> This data corresponds to the hardware configuration for computing GEMM ( $512 \times 768 \times 768$ ) with  $c = 32$ ,  $v = 4$ , codebook parallelism=1, and LUT bank=16.

<sup>b</sup> The proportion of effective MAC cycle in the overall cycle.

files and SRAMs) are generated by ARM memory compilers. All verilog codes are synthesized by Cadence Genus on the 28-nm FD-SOI node for area and power evaluation. We provide the parallelism parameters and bandwidth requirements of our design in Table VII, while Table VIII presents a comparison of the specific hardware data (PPA) of our work to other existing works. Since neither PIM-DL nor PECAN has specific hardware designs, we chose PQA for comparison. We replicated the operational procedure based on the architecture and dataflow proposed in the PQA paper and conducted the same GEMM operations as LUT-DLA under the same computational parallelism ( $v$ ,  $c$ ). Table IX presents the on-chip resources required and execution cycles for both architectures.

We also conduct runtime performance, area efficiency, and energy efficiency measurements on ResNet18 and BERT.

**Hardware Evaluation** In Fig. 14, the area costs of Design 1 is close to NVDLA-Small, but performance improves  $6.2 \times$

and  $12.0\times$  in BERT and ResNet18. In both tasks, the area efficiency of LUT-DLA is  $2.5\times/4.8\times$  higher than NVDLA-Small, and the energy efficiency is also  $1.1\times/4.01\times$  of the benchmark design.

Design 2 achieves throughput similar with NVDLA-Large, but its area is only  $\frac{1}{7}$ . Its area efficiencies on BERT and ResNet18 are  $14.6\times/10.7\times$  over NVDLA-Large, and the energy efficiency even increased by  $50.1\times/27.02\times$ . Furthermore, compared to Gemini, which has the same power consumption, Design 2 is  $3.5\times/7.8\times$  faster than Gemini, accompanied by an increase in area efficiency of  $4.1\times/9.02\times$  and energy efficiency of  $5.5\times/26.8\times$ .

Design 3 is an exploration result using our co-design engine that aims to maximize the BERT inference throughput. Its area and power consumption are only  $2/3$  of NVDLA-Large, but its throughput is  $2.3\times$  over NVDLA-Large. Its area efficiency and energy efficiency also exceed the benchmark design of  $11.3\times/86.8\times$ , which are even superior to Design 1 and 2.

Table VII further illustrates the hardware resources utilized by a single IMM implemented in each design.  $v$ ,  $N_c$ , and  $T_n$  are parameters in Algorithm 1, while  $M$  denotes the maximum number of rows in the input matrix tile during LUT-DLA's execution of GEMM. The minimum bandwidth for continuous IMM operation can be expressed as  $T_n \times N_c/M \times \text{Freq}$ , indicating that the computation time for  $M$  rows must cover LUT loading time for the next iteration. Increased bandwidth may be needed to conceal communication overhead if the workload tile row count is below  $M$ . Multiple IMM increase bandwidth usage, making the architecture memory-bound. To alleviate this, LUT-DLA provides an optional Global Memory for data buffering when bandwidth is inadequate.

**Compare with Other Designs** Table VIII shows a comparison of our work with recent accelerator designs. Gemini and NVDLA utilize the INT8 for computation; ELSA employs custom fixed-point formats, FACT implements a mixed precision (INT8 + INT4), while RRAM-DNN adopts INT8 computation optimized by Huffman coding. As shown in the table, LUT-DLA achieves a promising  $1.4\sim 7.0\times$  and  $1.5\sim 146.1\times$  improvement in power and area efficiency compared with recent DLA architectures.

Table IX presents the comparison results with other LUT-based accelerators PQA. The architectural design of PQA does not allow for data reuse, thereby requiring the loading of the entire layer parameters on-chip and causing a compute pause. Such design requires a lot of on-chip storage, longer operating time, and is difficult to scale. In contrast, LUT-DLA ensures on-demand data movement and parallel execution of computation pipelines and data loading. When performing the same GEMM under the same parameters, even though LUT-DLA employs ping-pong buffer, memory costs are significantly reduced and computation is  $1.6\times$  faster than PQA.

### C. End-to-end Performance

**Settings** Circuit-level simulations are time-consuming, we provide a cycle-accurate simulator for LUT-DLA. In the following experiments, NVDLA performance is calculated by

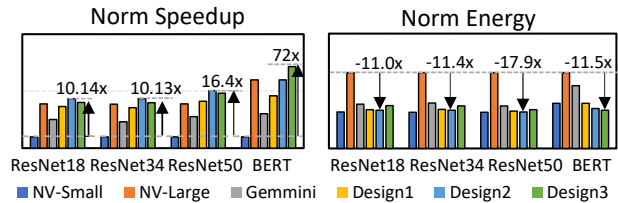


Fig. 13: End-to-End Throughput and Energy Consumption

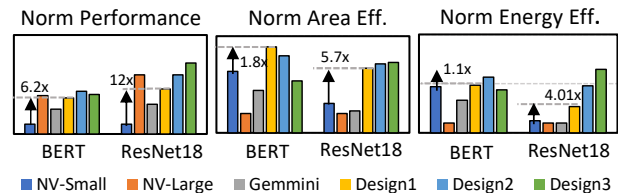


Fig. 14: PPA Analysis

the official performance model [44], Gemini cycle count [15] is simulated by Verilator. The performance of LUT-DLA is based on our cycle-accurate simulator, which assumes a maximum off-chip bandwidth of 25.6GB/s (DDR4). For all ResNet models, we collect the time cost for all convolution computations and linear layers. For Transformer models, we compute the time cost on computationally intensive operations (QKV Projection and FFN layers). We use the operating cycles among with PPA in Table VIII to simulate the energy consumption of model inference on different hardware designs.

**Results** Fig. 13 shows that LUT-DLA has advantages in a variety of scenarios. Design 2 outperforms the NVDLA-Large design in CNN (ResNet) models and saves  $11\times$  energy consumption. Although the performance of Design 1 is slightly lower than NVDLA-Large, its advantage lies in its compact area and lower power consumption. Our Design 3 achieves the best performance on BERT, achieving up to  $72\times$  performance improvement and  $11.5\times$  reduction in energy consumption, demonstrating the superiority of the LUT-DLA. LUT-DLA generally has overall higher area and energy efficiency scenarios, and these designs have been specifically optimized by the Co-design Engine to perform well in most designs.

## VIII. CONCLUSION

In this paper, we propose LUT-DLA, a LUT accelerator generator for LUT-based model inference on an emerging computational paradigm. Our experiments show that LUT-DLA shows promising  $1.4\sim 7.0\times$  and  $1.5\sim 146.1\times$  improvement in power and area efficiency compared to existing architectures with a minor accuracy drop. For CNNs, there is a slight decrease in accuracy between 0.1% and 3.1% when utilizing the  $L_2$  distance, a marginally higher drop ranging from 0.1% to 3.4% for the  $L_1$  distance, and a tolerable decrease from 0.1% to 3.8% with the Chebyshev distance. Transformer-based models also exhibit a diminutive accuracy decline between 1.4% and 3.0%.



## REFERENCES

- [1] A. F. AbouElhamayed, A. Cui, J. Fernandez-Marques, N. D. Lane, and M. S. Abdelfattah, "Pqa: Exploring the potential of product quantization in dnn hardware acceleration," *ACM Trans. Reconfigurable Technol. Syst.*, apr 2024, just Accepted. [Online]. Available: <https://doi.org/10.1145/3656643>
- [2] G. Armeniakos, G. Zervakis, D. Soudris, and J. Henkel, "Hardware approximate techniques for deep neural network accelerators: A survey," *ACM Comput. Surv.*, vol. 55, no. 4, nov 2022. [Online]. Available: <https://doi.org/10.1145/3527156>
- [3] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniek, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *DAC Design automation conference 2012*. IEEE, 2012, pp. 1212–1221.
- [4] D. W. Blalock and J. V. Guttat, "Multiplying matrices without multiplying," in *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 2021, pp. 992–1004. [Online]. Available: <http://proceedings.mlr.press/v139/blalock21a.html>
- [5] J. Burgess, "RTX on - the NVIDIA turing GPU," *IEEE Micro*, vol. 40, no. 2, pp. 36–44, 2020. [Online]. Available: <https://doi.org/10.1109/MM.2020.2971677>
- [6] X. Cai, Y. Wang, X. Ma, Y. Han, and L. Zhang, "Deepburning-seg: Generating DNN accelerators of segment-grained pipeline architecture," in *55th IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022*. IEEE, 2022, pp. 1396–1413. [Online]. Available: <https://doi.org/10.1109/MICRO56248.2022.00094>
- [7] J. Chee, Y. Cai, V. Kuleshov, and C. D. Sa, "QuIP: 2-bit quantization of large language models with guarantees," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: <https://openreview.net/forum?id=xrk9g5vcXR>
- [8] J. Choquette, "NVIDIA hopper H100 GPU: scaling performance," *IEEE Micro*, vol. 43, no. 3, pp. 9–17, 2023. [Online]. Available: <https://doi.org/10.1109/MM.2023.3256796>
- [9] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "NVIDIA A100 tensor core GPU: performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021. [Online]. Available: <https://doi.org/10.1109/MM.2021.3061394>
- [10] P. Cosman, K. Oehler, E. Riskin, and R. Gray, "Using vector quantization for image processing," *Proceedings of the IEEE*, vol. 81, no. 9, pp. 1326–1341, 1993.
- [11] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: accurate post-training quantization for generative pre-trained transformers," *CoRR*, vol. abs/2210.17323, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.17323>
- [12] A. Fuchs and D. Wentzlaff, "The accelerator wall: Limits of chip specialization," in *25th IEEE International Symposium on High Performance Computer Architecture, HPCA 2019, Washington, DC, USA, February 16-20, 2019*. IEEE, 2019, pp. 1–14. [Online]. Available: <https://doi.org/10.1109/HPCA.2019.00023>
- [13] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization for approximate nearest neighbor search," in *2013 IEEE Conference on Computer Vision and Pattern Recognition, Portland, OR, USA, June 23-28, 2013*. IEEE Computer Society, 2013, pp. 2946–2953. [Online]. Available: <https://doi.org/10.1109/CVPR.2013.379>
- [14] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 4, pp. 744–755, 2014.
- [15] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 769–774.
- [16] R. Gray, "Vector quantization," *IEEE ASSP Magazine*, vol. 1, no. 2, pp. 4–29, 1984.
- [17] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, Y. Solihin and
- M. A. Heinrich, Eds. ACM, 2023, pp. 3:1–3:15. [Online]. Available: <https://doi.org/10.1145/3579371.3589038>
- [18] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1414–1433.
- [19] S. Gupta, M. Imani, H. Kaur, and T. S. Rosing, "NNPIM: A processing in-memory architecture for neural network acceleration," *IEEE Trans. Computers*, vol. 68, no. 9, pp. 1325–1337, 2019. [Online]. Available: <https://doi.org/10.1109/TC.2019.2903055>
- [20] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 692–705. [Online]. Available: <https://doi.org/10.1109/ISCA2021.00060>
- [21] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," in *2013 18th IEEE European Test Symposium (ETS)*. IEEE, 2013, pp. 1–6.
- [22] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: efficient inference engine on compressed deep neural network," in *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 2016, pp. 243–254. [Online]. Available: <https://doi.org/10.1109/ISCA.2016.30>
- [23] A. Ho, E. Erdil, and T. Besiroglu, "Limits to the energy efficiency of cmos microprocessors," in *2023 IEEE International Conference on Rebooting Computing (ICRC)*. IEEE, 2023, pp. 1–10.
- [24] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," *CoRR*, vol. abs/2203.15556, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2203.15556>
- [25] M. Imani, M. Samragh, Y. Kim, S. Gupta, F. Koushanfar, and T. Rosing, "RAPIDNN: in-memory deep neural network acceleration framework," *CoRR*, vol. abs/1806.05794, 2018. [Online]. Available: <http://arxiv.org/abs/1806.05794>
- [26] M. Item, J. Gómez-Luna, Y. Guo, G. F. Oliveira, M. Sadrosadati, and O. Mutlu, "Transpimlib: A library for efficient transcendental functions on processing-in-memory systems," *CoRR*, vol. abs/2304.01951, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.01951>
- [27] N. P. Jouppi, G. Kurian, S. Li, P. C. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, Y. Solihin and M. A. Heinrich, Eds. ACM, 2023, pp. 82:1–82:14. [Online]. Available: <https://doi.org/10.1145/3579371.3589350>
- [28] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.
- [29] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.
- [30] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *CoRR*, vol. abs/2001.08361, 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [31] R. Landauer, "Irreversibility and heat generation in the computing process," *IBM J. Res. Dev.*, vol. 5, no. 3, pp. 183–191, 1961. [Online]. Available: <https://doi.org/10.1147/rd.53.0183>
- [32] C. Li, Z. Zhou, Y. Wang, F. Yang, T. Cao, M. Yang, Y. Liang, and G. Sun, "Pim-dl: Expanding the applicability of commodity dram-pims for deep learning via algorithm-system co-optimization," in *ASPLOS '24: 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 2024.
- [33] Y. Li, C. Zhang, S. Han, L. L. Zhang, B. Yin, Y. Liu, and M. Xu, "Boosting mobile CNN inference through semantic memory," in *MM '21: ACM Multimedia Conference, Virtual Event, China, October 20*

- 24, 2021, H. T. Shen, Y. Zhuang, J. R. Smith, Y. Yang, P. César, F. Metzger, and B. Prabhakaran, Eds. ACM, 2021, pp. 2362–2371. [Online]. Available: <https://doi.org/10.1145/3474085.3475399>
- [34] Z. Li, Z. Wang, L. Xu, Q. Dong, B. Liu, C. Su, W. Chu, G. Tsou, Y. Cih, T. J. Chang, D. Sylvester, H. Kim, and D. T. Blaauw, “RRAM-DNN: an RRAM and model-compression empowered all-weights-on-chip DNN accelerator,” *IEEE J. Solid State Circuits*, vol. 56, no. 4, pp. 1105–1115, 2021. [Online]. Available: <https://doi.org/10.1109/JSSC.2020.3045369>
- [35] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: activation-aware weight quantization for LLM compression and acceleration,” *CoRR*, vol. abs/2306.00978, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2306.00978>
- [36] S. Liu, Z. Liu, X. Huang, P. Dong, and K. Cheng, “LLM-FP4: 4-bit floating-point quantized transformers,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6–10, 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Association for Computational Linguistics, 2023, pp. 592–605. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.39>
- [37] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, “The era of 1-bit llms: All large language models are in 1.58 bits,” *CoRR*, vol. abs/2402.17764, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2402.17764>
- [38] J. Makhoul, S. Roucos, and H. Gish, “Vector quantization in speech coding,” *Proceedings of the IEEE*, vol. 73, no. 11, pp. 1551–1588, 1985.
- [39] P. Micikevicius, D. Stolic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. F. Oberman, M. Shoenybi, M. Y. Siu, and H. Wu, “FP8 formats for deep learning,” *CoRR*, vol. abs/2209.05433, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2209.05433>
- [40] P. Micikevicius, D. Stolic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. F. Oberman, M. Shoenybi, M. Y. Siu, and H. Wu, “FP8 formats for deep learning,” *CoRR*, vol. abs/2209.05433, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2209.05433>
- [41] L. Mocerino, V. Tenace, and A. Calimera, “Energy-efficient convolutional neural networks via recurrent data reuse,” in *Design, Automation Test in Europe Conference Exhibition, DATE 2019, Florence, Italy, March 25–29, 2019*, J. Teich and F. Fummi, Eds. IEEE, 2019, pp. 848–853. [Online]. Available: <https://doi.org/10.23919/DATE.2019.8714880>
- [42] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, “Evoapprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods,” in *Design, Automation Test in Europe Conference Exhibition (DATE), 2017*, 2017, pp. 258–261.
- [43] O. Mutlu, “Memory-centric computing,” *CoRR*, vol. abs/2305.20000, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.20000>
- [44] Nvidia. Nvdl open source hardware performance. Accessed on 2023-11-28. [Online]. Available: <https://github.com/nvidia/hw/blob/nvdlav1/perf>
- [45] NVIDIA. Nvidia deep learning accelerator. Accessed on 2023-11-13. [Online]. Available: <http://nvdl.org>
- [46] NVIDIA. (2024) Nvidia dgx b200 datasheet. [Online]. Available: <https://resources.nvidia.com/en-us-dgx-systems/dgx-b200-datasheet>
- [47] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. S. Emer, S. W. Keckler, and W. J. Dally, “SCNN: an accelerator for compressed-sparse convolutional neural networks,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24–28, 2017*. ACM, 2017, pp. 27–40. [Online]. Available: <https://doi.org/10.1145/3079856.3080254>
- [48] G. Park, B. Park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, “LUT-GEMM: quantized matrix multiplication based on luts for efficient inference in large-scale generative language models,” in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=gLARhFLE0F>
- [49] Y. Qin, Y. Wang, D. Deng, Z. Zhao, X. Yang, L. Liu, S. Wei, Y. Hu, and S. Yin, “FACT: fn-attention co-optimized transformer architecture with eager correlation prediction,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17–21, 2023*, Y. Solihin and M. A. Heinrich, Eds. ACM, 2023, pp. 22:1–22:14. [Online]. Available: <https://doi.org/10.1145/3579371.3589057>
- [50] J. Ran, R. Lin, J. C. L. Li, J. Zhou, and N. Wong, “PECAN: A product-quantized content addressable memory network,” in *Design, Automation Test in Europe Conference Exhibition, DATE 2023, Antwerp, Belgium, April 17–19, 2023*. IEEE, 2023, pp. 1–6. [Online]. Available: <https://doi.org/10.23919/DATE56975.2023.10137218>
- [51] M. Riera, J. Arnau, and A. González, “Computation reuse in dnns by exploiting input similarity,” in *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1–6, 2018*, M. Annavaram, T. M. Pinkston, and B. Falsafi, Eds. IEEE Computer Society, 2018, pp. 57–68. [Online]. Available: <https://doi.org/10.1109/ISCA.2018.00016>
- [52] J. Schönleber, L. Cavigelli, R. Andri, M. Perotti, and L. Benini, “Stella nera: Achieving 161 top/s/w with multiplier-free DNN acceleration based on approximate matrix multiplication,” *CoRR*, vol. abs/2311.10207, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2311.10207>
- [53] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, “Softmax: Hardware/software co-design of an efficient softmax for transformers,” in *58th ACM/IEEE Design Automation Conference, DAC 2021, San Francisco, CA, USA, December 5–9, 2021*. IEEE, 2021, pp. 469–474. [Online]. Available: <https://doi.org/10.1109/DAC18074.2021.9586134>
- [54] A. Stillmaker and B. M. Baas, “Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm,” *Integr.*, vol. 58, pp. 74–81, 2017. [Online]. Available: <https://doi.org/10.1016/j.vlsi.2017.02.002>
- [55] X. Tang, Y. Wang, T. Cao, L. L. Zhang, Q. Chen, D. Cai, Y. Liu, and M. Yang, “LUT-NN: empower efficient neural network inference with centroid learning and table lookup,” in *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking, ACM MobiCom 2023, Madrid, Spain, October 2–6, 2023*, X. Costa-Pérez, J. Widmer, D. Perino, D. Giustiniano, H. Al-Hassanieh, A. Asadi, and L. P. Cox, Eds. ACM, 2023, pp. 70:1–70:15. [Online]. Available: <https://doi.org/10.1145/3570361.3613285>
- [56] Z.-G. Tasoulas, G. Zervakis, I. Anagnostopoulos, H. Amrouch, and J. Henkel, “Weight-oriented approximation for energy-efficient neural network inference accelerators,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4670–4683, 2020.
- [57] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. D. Sa, “Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks,” 2024.
- [58] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, T. Linzen, G. Chrupala, and A. Alishahi, Eds. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355. [Online]. Available: <https://aclanthology.org/W18-5446>
- [59] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, “Bitnet: Scaling 1-bit transformers for large language models,” *CoRR*, vol. abs/2310.11453, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.11453>
- [60] L. Wang, X. Dong, Y. Wang, L. Liu, W. An, and Y. Guo, “Learnable lookup table for neural network quantization,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18–24, 2022*. IEEE, 2022, pp. 12 413–12 423. [Online]. Available: <https://doi.org/10.1109/CVPR52688.2022.01210>
- [61] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, “Nn-lut: Neural approximation of non-linear operations for efficient transformer inference,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 577–582. [Online]. Available: <https://doi.org/10.1145/3489517.3530505>
- [62] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W. W. Hwu, and D. Chen, “Dnnbuilder: an automated tool for building high-performance DNN hardware accelerators for fpgas,” in *Proceedings of the International Conference on Computer-Aided Design, ICCAD 2018, San Diego, CA, USA, November 05–08, 2018*, I. Bahar, Ed. ACM, 2018, p. 56. [Online]. Available: <https://doi.org/10.1145/3240765.3240801>